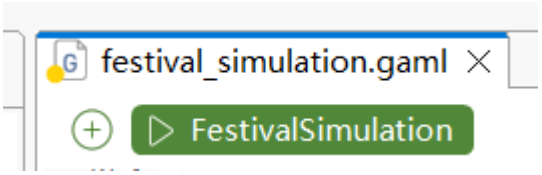
ID2209 Assignment 3: N Queens & Utility Function

Group 3
Shunkang Jia, Rui Zhang, Yuning Liu
shunkang, rzha, yuningl @kth.se

Section 1: Running Instruction

Copy the*.gaml file into user model section and click the “FestivalSimulation” button.
Each task has its own gaml file, overall 3 files are submitted.



Section 2: General Overview

In this assignment we have finished N queen simulation and implemented a utility function for agents simulating their preference in Stages. We have also conquered challenge 1, designing a leader agent to optimize the global utility function by hill climbing.

Section 3: N Queen Problem

3.1 Explanation

This section implements a distributed N-Queens solver, where each queen is modeled as an independent agent responsible for placing itself on a fixed row of an N x N chessboard. The agents cooperate using a message-passing mechanism rather than direct actions such as ask.

Model Overview

- There are N queen agents, each assigned to a unique row 0..N-1.
- A queen attempts to choose a valid column that does not conflict with:
  - any previous queen’s column
  - or diagonal positions
- When a queen cannot find any valid column:
  - she sends a BACKTRACK message to her predecessor
  - the predecessor then tries its next available column
- When a queen successfully selects a column:
  - she sends a POSITION_REQUEST message to her successor
  - the successor continues the search
- When the last queen (row N−1) finds a valid column:
  - a complete solution is recorded

· if more solutions are required, a new BACKTRACK cycle is initiated

Message Protocol

We implement a lightweight custom message system with three message types:

Message	Sender	Receiver	Meaning
POSITION_REQUEST	queen *i*	queen *i+1*	Try to place yourself given current partial board
BACKTRACK	queen *i*	queen *i-1*	I have no valid position—please change your position
RESET	queen *i*	queen *i+1*	Clear your state because I will retry

These messages are stored in each queen's own fields (`has_message`, `msg_type`, `msg_data`) and processed asynchronously inside a reflex. This ensures that no queen directly manipulates another agent's internal variables, maintaining the distributed nature of the system.

Solution Search Logic

The solver uses depth-first backtracking implemented in a distributed manner:

1. Queen 0 starts by sending `"START"` to itself.
2. Each queen:
   · collects the "occupied columns" of all earlier rows
   · finds the next valid column
3. If a valid column is found:
   · store position
   · forward the partial solution to successor
4. If the queen has no valid column:
   · send `"BACKTRACK"` to predecessor
5. When the last queen succeeds:
   · print the full solution
   · increment solution counter
   · if more solutions are required → trigger new backtracking round
6. The search terminates when the number of required solutions has been reached.

3.2 Code

3.2.1 Queen Agent Definition & Message Structure

Each queen stores its own row, current column choice, attempted columns, and a message buffer. The reflex `process_message` enforces asynchronous communication.

```
species queen_agent {
    int queen_id;
    int row;
    int n;
    int position <- -1;
    list<int> tried_positions <- [];
    queen_agent predecessor <- nil;
    queen_agent successor <- nil;
    bool is_placed <- false;

    bool has_message <- false;
    string msg_type <- "";
    list<int> msg_data <- [];
```

3.2.2 Message-Based Distributed Backtracking

When a queen receives a `"POSITION_REQUEST"` message, she tries the next valid column:

· Checks column and diagonal safety
· If valid → store and forward to successor
· If invalid or exhausted → send `"BACKTRACK"` to predecessor

When receiving `"BACKTRACK"`:

· Reset successor chain
· Compute remaining valid positions
· Retry placement

```
reflex process_message when: has_message {
    has_message <- false;

    if msg_type = "START" or msg_type = "POSITION_REQUEST" {
        list<int> occupied <- copy(msg_data);
        int next_pos <- find_next_position(occupied);

        if next_pos >= 0 {
            position <- next_pos;
            is_placed <- true;
            tried_positions <- tried_positions + next_pos;

            write "Queen " + queen_id + " placed at column " + next_pos;

            if successor != nil {
                list<int> new_occupied <- copy(occupied) + next_pos;
                successor.has_message <- true;
                successor.msg_type <- "POSITION_REQUEST";
                successor.msg_data <- copy(new_occupied);
            } else {
                solutions_found <- solutions_found + 1;
                write "\n=== SOLUTION " + solutions_found + " ===";

                list<queen_agent> all_q <- list(queen_agent);
                loop q over: all_q {

                loop r from: 0 to: n - 1 {
                write "";

                if solutions_found < max_solutions {
                    write "Finding solution " + (solutions_found + 1) + "...\n";
                    has_message <- true;
                    msg_type <- "BACKTRACK";
                    msg_data <- [];
                } else {
            }
        } else {
            write "Queen " + queen_id + " backtracking";
            if predecessor != nil {
                predecessor.has_message <- true;
                predecessor.msg_type <- "BACKTRACK";
                predecessor.msg_data <- [];
            }
```

```
else if msg_type = "BACKTRACK" {
    if successor != nil {
        successor.has_message <- true;
        successor.msg_type <- "RESET";
        successor.msg_data <- [];
    }

    list<int> occupied <- [];
    queen_agent temp <- predecessor;
    loop while: temp != nil {
        occupied <- [temp.position] + occupied;
        temp <- temp.predecessor;
    }

    int next_pos <- find_next_position(occupied);

    if next_pos >= 0 {
        position <- next_pos;
        is_placed <- true;
        tried_positions <- tried_positions + next_pos;

        write "Queen " + queen_id + " placed at column " + next_pos;

        if successor != nil {
            list<int> new_occupied <- copy(occupied) + next_pos;
            successor.has_message <- true;
            successor.msg_type <- "POSITION_REQUEST";
            successor.msg_data <- copy(new_occupied);
        } else {
            solutions_found <- solutions_found + 1;
            write "\n=== SOLUTION " + solutions_found + " ===";

            list<queen_agent> all_q <- list(queen_agent);
            loop q over: all_q {

            loop r from: 0 to: n - 1 {
            write "";

            if solutions_found < max_solutions {
        }
    } else {
        position <- -1;
        is_placed <- false;
```

```
            } else {
                position <- -1;
                is_placed <- false;
                tried_positions <- [];
                if predecessor != nil {
                    predecessor.has_message <- true;
                    predecessor.msg_type <- "BACKTRACK";
                    predecessor.msg_data <- [];
                }
            }
        }
        else if msg_type = "RESET" {
            position <- -1;
            is_placed <- false;
            tried_positions <- [];
            if successor != nil {
                successor.has_message <- true;
                successor.msg_type <- "RESET";
                successor.msg_data <- [];
            }
        }
    }
```

3.2.3 Solution Collection

When the last queen (row = N−1) finds a valid column:

· The system prints the current full board

· Records the solution in `solutions_found`

· If more solutions are needed → start a new `"BACKTRACK"` process

· If target reached → stop simulation

```
        solutions_found <- solutions_found + 1;
        write "\n=== SOLUTION " + solutions_found + " ===";

        list<queen_agent> all_q <- list(queen_agent);
        loop q over: all_q {
            write "Queen " + q.queen_id + " at column " + q.position;
        }

        loop r from: 0 to: n - 1 {
            string row_str <- "";
            loop c from: 0 to: n - 1 {
                bool found <- false;
                loop q over: all_q {
                    if q.row = r and q.position = c {
                        found <- true;
                    }
                }
                row_str <- row_str + (found ? "Q " : ". ");
            }
            write row_str;
        }
        write "";

        if solutions_found < max_solutions {
            write "Finding solution " + (solutions_found + 1) + "...\n";
            has_message <- true;
            msg_type <- "BACKTRACK";
            msg_data <- [];
        } else {
            write "=== COMPLETE: " + solutions_found + " SOLUTIONS ===";
        }
    }
```

3.3 Demonstration

3.3.1 Use case 1 —- Correct Solutions for N=4

(1) Input Description

We run the model with:

· board_size = 4

· max_solutions = 2

Expected output: the two known solutions for 4-queens.

(2) Screenshot

```
Queen 0 placed at column 0
Queen 1 placed at column 2
Queen 2 backtracking
Queen 1 placed at column 3
Queen 2 placed at column 1
Queen 3 backtracking
Queen 0 placed at column 1
Queen 1 placed at column 3
Queen 2 placed at column 0
Queen 3 placed at column 2

=== SOLUTION 1 ===
Queen 0 at column 1
Queen 1 at column 3
Queen 2 at column 0
Queen 3 at column 2
. Q . .
. . . Q
Q . . .
. . Q .

Finding solution 2...

Queen 0 placed at column 2
Queen 1 placed at column 0
Queen 2 placed at column 3
Queen 3 placed at column 1

=== SOLUTION 2 ===
Queen 0 at column 2
Queen 1 at column 0
Queen 2 at column 3
Queen 3 at column 1
. . Q .
Q . . .
. . . Q
. Q . .

=== COMPLETE: 2 SOLUTIONS ===
```

(3) Interpretation

The system successfully outputs the two classical N=4 arrangements:

Solution 1: [1,3,0,2]
Solution 2: [2,0,3,1]
The logs show queens communicating strictly via the message system. Backtracking is triggered exactly when a queen fails to find an available column.
This confirms that the distributed message-passing backtracking process is functioning correctly.

3.3.2 Use case 2 —- Correct Solution for N = 8

(1) Input Description
To demonstrate scalability, we run:
· board_size = 8
· max_solutions = 1

(2) Screenshot

```
Queen 6 backtracking
Queen 4 placed at column 5
Queen 5 placed at column 2
Queen 6 backtracking
Queen 5 placed at column 7
Queen 6 backtracking
Queen 2 placed at column 7
Queen 3 placed at column 1
Queen 4 placed at column 3
Queen 5 placed at column 6
Queen 6 placed at column 2
Queen 7 backtracking
Queen 4 placed at column 6
Queen 5 placed at column 2
Queen 6 placed at column 5
Queen 7 backtracking
Queen 3 placed at column 5
Queen 4 placed at column 2
Queen 5 placed at column 6
Queen 6 placed at column 1
Queen 7 placed at column 3

=== SOLUTION 1 ===
Queen 0 at column 0
Queen 1 at column 4
Queen 2 at column 7
Queen 3 at column 5
Queen 4 at column 2
Queen 5 at column 6
Queen 6 at column 1
Queen 7 at column 3
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
```

```
Queen 2 placed at column 7
Queen 3 placed at column 1
Queen 4 placed at column 3
Queen 5 placed at column 6
Queen 6 placed at column 2
Queen 7 backtracking
Queen 4 placed at column 6
Queen 5 placed at column 2
Queen 6 backtracking
Queen 3 placed at column 2
Queen 4 placed at column 6
Queen 5 placed at column 3
Queen 6 placed at column 1
Queen 7 placed at column 4

=== SOLUTION 2 ===
Queen 0 at column 0
Queen 1 at column 5
Queen 2 at column 7
Queen 3 at column 2
Queen 4 at column 6
Queen 5 at column 3
Queen 6 at column 1
Queen 7 at column 4
Q . . . . . . .
. . . . . Q . .
. . . . . . . Q
. . Q . . . . .
. . . . . . Q .
. . . Q . . . .
. Q . . . . . .
. . . . Q . . .

=== COMPLETE: 2 SOLUTIONS ===
```

(3) Interpretation

The solver correctly finds one valid solution for N=8. The number of backtracking events increases significantly due to the exponential search space, which matches expectations. This shows the solution generalizes to larger N values.

Section 4: Task 2 Positioning Speakers at Main Stage

4.1 Explanation

Our simulation models a dynamic festival environment where **50** guest agents actively choose between **4** stage agents. The simulation focuses on agent decision-making based on multi-criteria utility function and FIPA communication.

We have following assumptions

- Guests always select the stage yielding the highest utility score based on their personal preference weights.
- Stages are not static; their attributes (Light, Sound, Visual, Music) fluctuate periodically (every 500 cycles).
- The show in every stage happens at the same time in each iteration. The number of stages are constant meaning there will not be any pop-up stages.
- A Guest will not move until it has received and processed data from *every* available stage to ensure a fair comparison.

We have also made a map visualization and guest count for each stage.

4.2 Code

4.2.1. Global Context

- **all_stages**: A global list storing references to all Stage agents. This allows Guests to verify they have received replies from every source before deciding.
- **change_interval**: Defines the rhythm of the simulation (set to 500 cycles), triggering Stage randomization and Guest re-evaluation simultaneously.

```
global {
    // 1. Global Setup
    This definition of shape supersedes the one in built-in species agent
    geometry shape <- square(100 #m);
    int change_interval <- 500;

    // List of stages for guests to check count against
    list<Stage> all_stages;

    init {
        // Create 4 Stages at corners
        create Stage number: 1 { location <- {0, 0}; name <- "Stage A"; my_color <- #red; }
        create Stage number: 1 { location <- {100, 0}; name <- "Stage B"; my_color <- #blue; }
        create Stage number: 1 { location <- {100, 100}; name <- "Stage C"; my_color <- #green; }
        create Stage number: 1 { location <- {0, 100}; name <- "Stage D"; my_color <- #purple; }

        // Store reference for guests
        all_stages <- list(Stage);

        // Create 50 Guests
        create Guest number: 50;
    }
}
```

4.2.2. Species: Stage

- **randomize_attributes**: Every 500 cycles, assigns random float values (0.0–1.0) to the stage's performance qualities (Lighting, Sound, Visuals, Music).

```
species Stage skills: [fipa] {
    // 2. Species: Stage
    float lighting_q;
    float sound_q;
    float visual_q;
    float music_q;
    rgb my_color;

    init {
        do randomize_attributes;
    }

    // Dynamic Reflex: Randomize attributes periodically
    reflex update_attributes when: (cycle > 0) and (cycle mod change_interval = 0) {
        do randomize_attributes;
        write name + ": Attributes shuffled!";
    }

    action randomize_attributes {
        lighting_q <- rnd(0.0, 1.0);
        sound_q <- rnd(0.0, 1.0);
        visual_q <- rnd(0.0, 1.0);
        music_q <- rnd(0.0, 1.0);
    }
```

- **reply_to_guests (Reflex):** Listens for incoming FIPA request messages. It packages the current attributes into a Map and sends an inform message back to the requester.

```
// Communication: Reply to guests
reflex reply_to_guests {
    // Constraint: Loop over requests
    loop req over: requests {
        // Prepare data map
        map<string, float> specs <- [
            "light"::lighting_q,
            "sound"::sound_q,
            "visual"::visual_q,
            "music"::music_q
        ];

        // Reply using 'inform'. GAML wraps 'contents' in a list automatically.
        do inform message: req contents: [specs];
    }
}
```

4.2.3. Species: Guest

- **Attributes (w_light, etc.)**: Randomly assigned weights representing the guest's personality (e.g., a guest who prioritizes "Sound" over "Visuals").

```
species Guest skills: [moving, fipa] {
    // 3. Species: Guest

    // Preferences (Weights)
    float w_light <- rnd(0.0, 1.0);
    float w_sound <- rnd(0.0, 1.0);
    float w_visual <- rnd(0.0, 1.0);
    float w_music <- rnd(0.0, 1.0);

    Stage target_stage <- nil;

    // Temporary memory to store utilities from stages before deciding
    map<Stage, float> stage_evaluations;
    bool waiting_for_decision <- false;
```

- **query_stages**: Triggers at the start and every change_interval. It broadcasts a FIPA request to all Stage agents in the global all_stages list.

```
// Step 1: Trigger Request
reflex query_stages when: (cycle = 0) or (cycle mod change_interval = 0) {
    // Reset state
    target_stage <- nil;
    stage_evaluations <- [];
    waiting_for_decision <- true;

    // Send request to ALL stages
    do start_conversation to: list(Stage) protocol: 'no-protocol' performative: 'request' contents: ['get_specs'];
}
```

- **collect_replies**: The core logic reflex. It:
    1. Loops through incoming inform messages.

2. **Calculates Utility** using a Weighted Sum Model
3. **Validates** that the number of received evaluations equals the total number of stages.

```
// Step 2: Receive & Parse
reflex collect_replies when: !empty(informs) and waiting_for_decision {

    // Constraint: Loop over informs
    loop msg over: informs {

        // --- Constraint: Exact Parsing Logic ---
        // 1. Extract content as list
        list repContents <- msg.contents;
        // 2. Cast first element to map
        ⊗Elements of specs are of type float but are assigned elements of
        map<string, float> specs <- map(repContents[0]);

        // Calculate Utility
        float utility <- (specs["light"] * w_light) +
                         (specs["sound"] * w_sound) +
                         (specs["visual"] * w_visual) +
                         (specs["music"] * w_music);

        // Store result mapping the sender (Stage agent) to the score
        ⊗The type of the index of stage_evaluations (Stage) does not match
        add utility at: (agent(msg.sender)) to: stage_evaluations;
    }

    // Step 4: Validation (Synchronization)
    // Check if we have received replies from ALL stages
    if (length(stage_evaluations) = length(all_stages)) {
        do make_decision;
    }
}
```

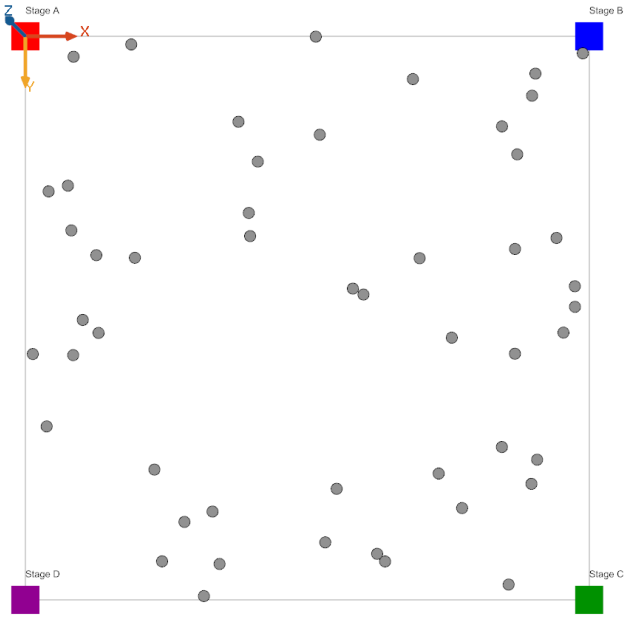
- **make_decision**: Finds the stage with the highest calculated utility, sets it as the target, and updates the agent's color.

```
// Step 5: Action
action make_decision {
    // Find the stage with the highest utility score
    float max_score <- -1.0;
    Stage best_stage <- nil;

    loop s over: stage_evaluations.keys {
        if (stage_evaluations[s] > max_score) {
            max_score <- stage_evaluations[s];
            best_stage <- s;
        }
    }

    target_stage <- best_stage;
    waiting_for_decision <- false; // Stop listening until next interval
}
```

4.3 Demonstration

4.3.1 Initial Setting

There are 4 stages placed in the 4 corners. Guest are randomly generated in the map. There are a total of 50 guest agents.



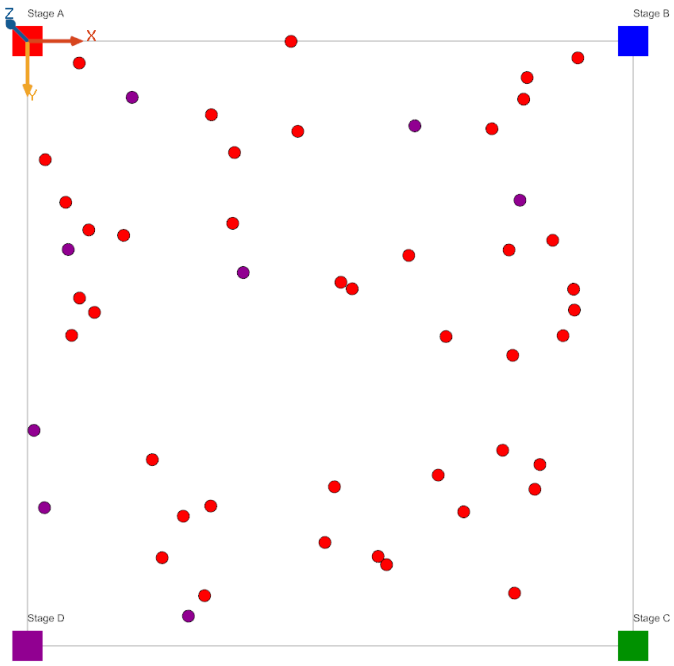
4.3.2 Specific Guest ID 42

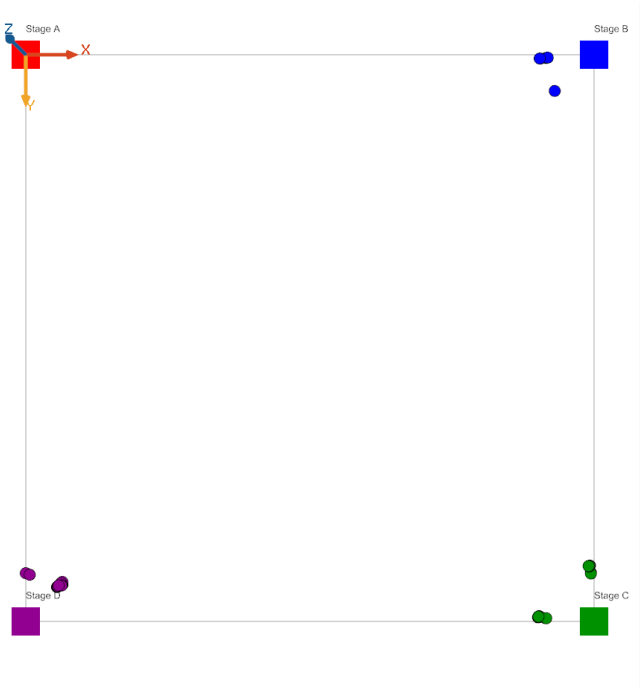
We select a specific guest and print his behavior logs.

```
--------------------------------------------------
 Guest 42 Analysis | Cycle: 1
   > My Weights: Light=0.08 | Sound=0.63 | Visual=0.68 | Music=0.63
   > Stage Evaluations:
     - Stage(0): 0.65
     - Stage(1): 1.25
     - Stage(2): 0.89
     - Stage(3): 1.48
 FINAL DECISION: Stage(3)
   > Winning Score: 1.48
--------------------------------------------------
Stage A: Attributes shuffled!
Stage B: Attributes shuffled!
Stage C: Attributes shuffled!
Stage D: Attributes shuffled!
--------------------------------------------------
 Guest 42 Analysis | Cycle: 501
   > My Weights: Light=0.08 | Sound=0.63 | Visual=0.68 | Music=0.63
   > Stage Evaluations:
     - Stage(0): 0.73
     - Stage(1): 1.39
     - Stage(2): 1.43
     - Stage(3): 1.36
 FINAL DECISION: Stage(2)
   > Winning Score: 1.43
--------------------------------------------------
Stage A: Attributes shuffled!
Stage B: Attributes shuffled!
Stage C: Attributes shuffled!
Stage D: Attributes shuffled!
--------------------------------------------------
 Guest 42 Analysis | Cycle: 1001
   > My Weights: Light=0.08 | Sound=0.63 | Visual=0.68 | Music=0.63
   > Stage Evaluations:
     - Stage(0): 0.82
     - Stage(1): 1.02
     - Stage(2): 0.97
     - Stage(3): 0.86
 FINAL DECISION: Stage(1)
   > Winning Score: 1.02
```

4.3.3 Global map visualization

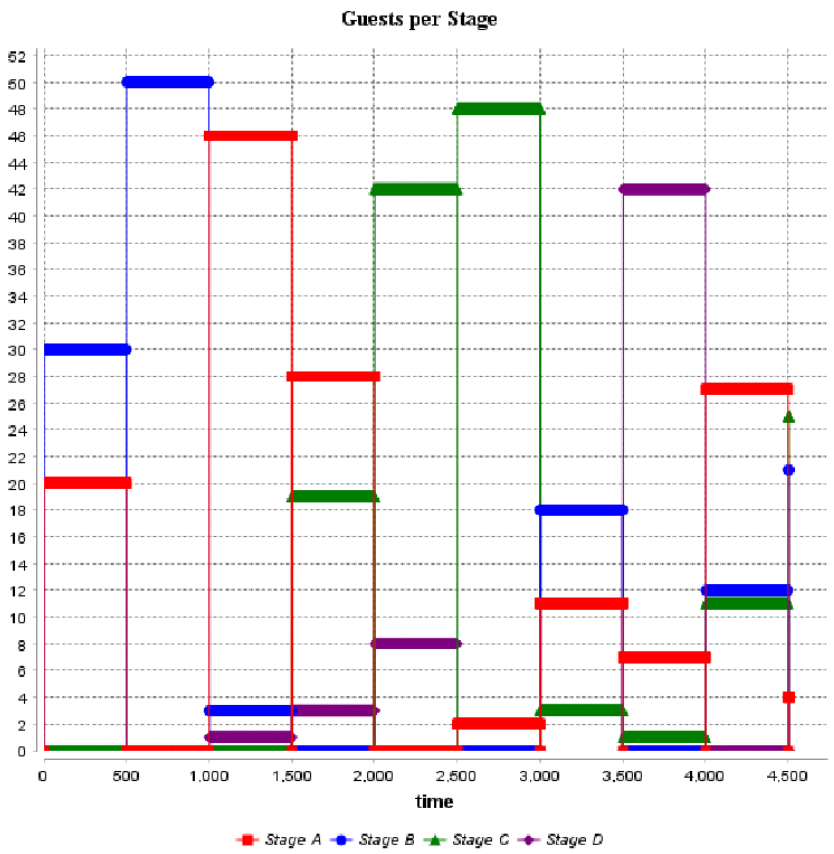
After guests have made decisions, they change color as their desired stage and go to it.





4.3.4 Guest Count of Stages

The line graph depicts the number of guests at each stage eversy timestamp .



Section 5 : Challenge 1 Global Utility Function

5.1 Explanation

In the previous iteration, agents acted "selfishly," moving to the stage with the best attributes regardless of overcrowding. This extension introduces a global utility function. Agents now possess a crowd_preference trait, ranging from **-1.0** (Hates Crowds) to **1.0** (Loves Crowds).

To manage these conflicting desires, we introduce a **Leader Agent**. Instead of moving immediately, Guests act as sensors, submitting their utility data to the Leader.

The Leader acts as a central optimizer, processing all 50 requests to calculate a configuration that maximizes **Global Utility** defined as:

$$GlobalU = \sum(BaseUtility + (CrowdPreference \times NormalizedCrowdSize))$$

Since calculating every permutation for 50 agents is computationally impossible (4^50 combinations), the Leader uses a Hill Climbing Heuristic:

1. Initially assign everyone to their favorite stage.

2. Randomly swap guests between stages. If the Global Utility increases, keep the swap; otherwise, revert.

5.2 Code

5.2.1. Species: AgentLeader

- **guest_data**: A map storing every guest's evaluation of the 4 stages and their specific crowd preference. The leader starts optimization only after he has received evaluations from all 50 agents.

```
//  Leader Species
species AgentLeader skills: [fipa] {
    // Data storage for the current optimization round
    // Map: Guest Agent -> [Map<Stage, float> base_scores, float crowd_pref]
    map<agent, list> guest_data <- [];
```

- **receive_guest_preferences:** Collect guest reports and store in the guest_data

```
reflex receive_guest_preferences when: !empty(informs) {
    loop msg over: informs {
        // Parse content: [stage_evaluations_map, crowd_preference_float]
        list content <- msg.contents;

        // Store data mapped to the sender (the Guest)
        add content at: agent(msg.sender) to: guest_data;
    }
}
```

- **optimize_and_assign (Reflex):** The core logic. It triggers only when all 50 guests have reported. It runs a loop (200 iterations) trying to optimize the crowd distribution to raise the global_utility_score.

```
// Optimize Global Utility
reflex optimize_and_assign when: length(guest_data) = 50 {
    write "Leader: Received all data. Optimizing...";

    // 1. Initial Assignment: Assign everyone to their favorite "Base" stage (Greedy initialization)
    temp_assignments <- [];
    loop g over: guest_data.keys {
        map<Stage, float> evaluations <- guest_data[g][0];

        // Find stage with max base score
        Stage best_s <- nil;
        float max_v <- -9999.0;
        loop s over: evaluations.keys {
            if (evaluations[s] > max_v) {
                max_v <- evaluations[s];
                best_s <- s;
            }
        }
        add best_s at: g to: temp_assignments;
    }

    // 2. Iterative Improvement (Hill Climbing)
    // Try to move guests to improve Global Utility
    int iterations <- 0;
    loop times: 200 { // 200 optimization steps
        agent candidate_guest <- one_of(guest_data.keys);
        Stage current_s <- temp_assignments[candidate_guest];
        Stage new_s <- one_of(all_stages - current_s);

        // Calculate Global Utility BEFORE swap
        float score_before <- calculate_global_utility();

        // Temporarily Swap
        add new_s at: candidate_guest to: temp_assignments;

        // Calculate Global Utility AFTER swap
        float score_after <- calculate_global_utility();

        // If new score is worse, revert the swap
        if (score_after <= score_before) {
             add current_s at: candidate_guest to: temp_assignments;
        }
    }

    // Update Global Variable for Charting
    global_utility_score <- calculate_global_utility();

    // Calculate Base Utility (What they would have got without crowd factor) for comparison
    base_utility_score <- 0.0;
    loop g over: guest_data.keys {
         map<Stage, float> evaluations <- guest_data[g][0];
         pref of type float is assigned a value of type unknown, which will be casted to float
         float pref <- guest_data[g][1]; // crowd pref
         Stage assigned <- temp_assignments[g];
         // Simple sum of base scores + crowd pref (raw)
         base_utility_score <- base_utility_score + (evaluations[assigned] + pref);
    }

    // 3. Send Orders (Assignments) to Guests via FIPA 'propose'
    loop g over: temp_assignments.keys {
        do start_conversation to: [g] protocol: 'fipa-propose' performative: 'propose' contents: [temp_assignments[g]];
    }

    // 4. Clear data for next cycle
    guest_data <- [];
    temp_assignments <- [];
    write "Leader: Assignments sent. Cycle complete.";
}
```

- **calculate_global_utility**: A helper function that calculates the sum of utilities for a specific configuration. It dynamically calculates the crowd size at each stage to apply the crowd preference penalties/bonuses.

```
// Helper function to calculate total utility of current configuration
float calculate_global_utility {
    float total_u <- 0.0;

    // Pre-calculate crowd counts for efficiency
    map<Stage, int> counts <- [];
    loop s over: all_stages { counts[s] <- 0; }
    loop g over: temp_assignments.keys {
        counts[temp_assignments[g]] <- counts[temp_assignments[g]] + 1;
    }

    // Sum Utilities
    loop g over: temp_assignments.keys {
        map<Stage, float> evaluations <- guest_data[g][0]; // Base scores
        float c_pref <- float(guest_data[g][1]); // Crowd pref
        Stage s <- temp_assignments[g];

        float base <- evaluations[s];

        // Normalized Crowd Size (0.0 to 1.0)
        float norm_crowd <- counts[s] / 50.0;

        // Formula: Base + (CrowdPref * NormalizedCrowd)
        float u <- base + (c_pref * norm_crowd);
        total_u <- total_u + u;
    }
    return total_u;
}
```

5.2.2. Species: Guest

Modifications are made to add crowd_preference feature.

- **crowd_preference**: A new random attribute (-1.0 to 1.0).

```
species Guest skills: [moving, fipa] {
  float w_light <- rnd(0.0, 1.0);
  float w_sound <- rnd(0.0, 1.0);
  float w_visual <- rnd(0.0, 1.0);
  float w_music <- rnd(0.0, 1.0);

  // -1.0 (Hates crowds) to 1.0 (Loves crowds)
  float crowd_preference <- rnd(-1.0, 1.0);

  Stage target_stage <- nil;
  map<Stage, float> stage_evaluations;
  bool waiting_for_specs <- false;
```

- **collect_replies (Modified):** Instead of moving immediately after calculating utilities, the agent now bundles the stage_evaluations map and the crowd_preference and sends them to the the_leader agent.

```
reflex collect_replies when: !empty(informs) and waiting_for_specs {
  loop msg over: informs {
    // Only process messages from Stages (ignore Leader logic here if mixed)
    if (msg.sender is Stage) {
      list repContents <- msg.contents;
      ⊗ Elements of specs are of type float but are assigned elements of type un
      map<string, float> specs <- map(repContents[0]);

      float utility <- (specs["light"] * w_light) +
                       (specs["sound"] * w_sound) +
                       (specs["visual"] * w_visual) +
                       (specs["music"] * w_music);

      ⊗ The type of the index of stage_evaluations (Stage) does not match with t
      add utility at: (agent(msg.sender)) to: stage_evaluations;
    }
  }

  // --- CHALLENGE 1: Report to Leader ---
  // Once we have evaluated all 4 stages, send data to Leader instead of moving
  if (length(stage_evaluations) = length(all_stages)) {
    waiting_for_specs <- false; // Done listening to stages

    // Send map of scores AND crowd preference to the Leader
    do start_conversation to: [the_leader]
                protocol: 'no-protocol'
                performative: 'inform'
                contents: [stage_evaluations, crowd_preference];
  }
}
```

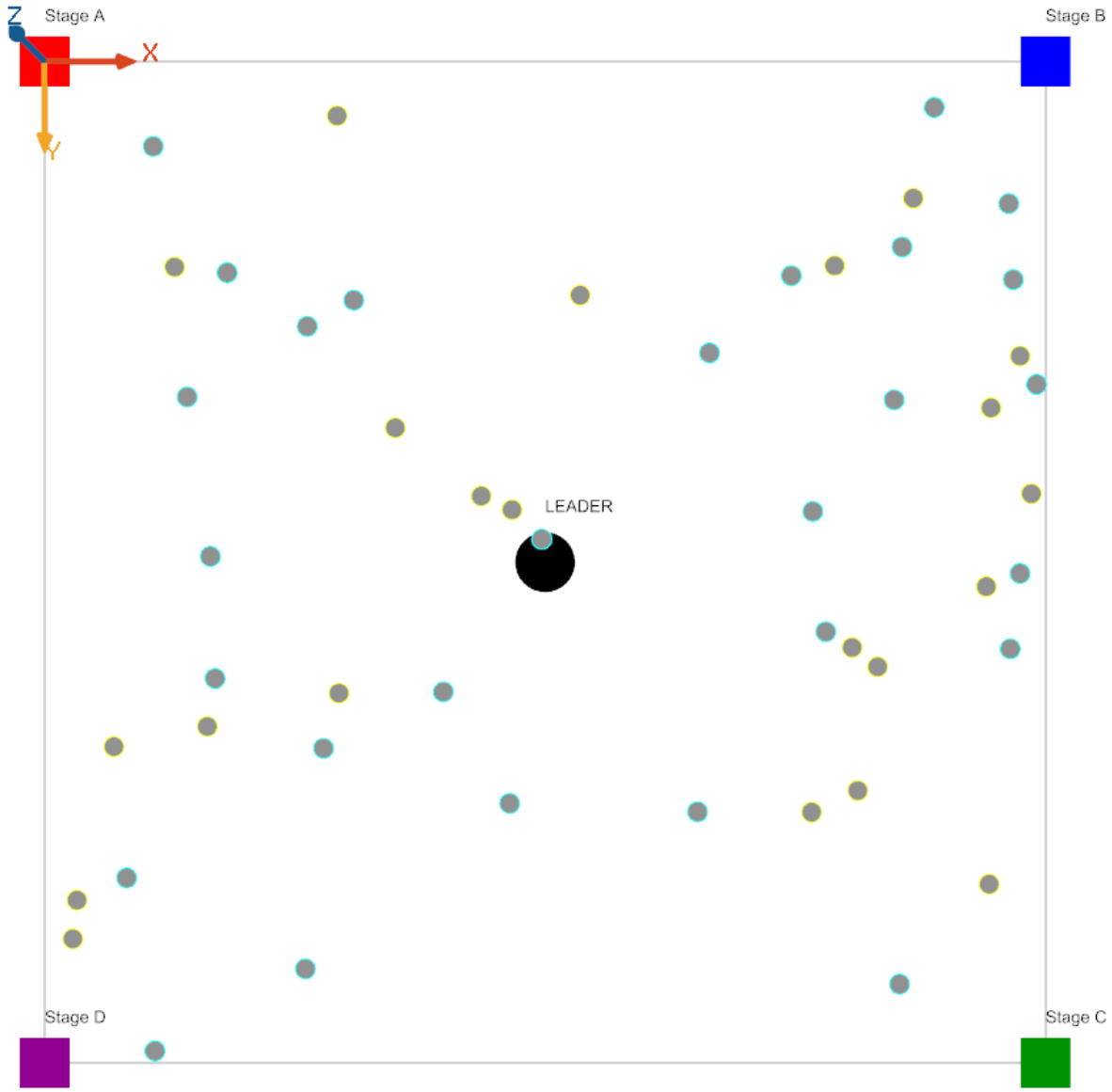
- **receive_orders (Reflex):** A new listener that waits for the Leader's propose message to update its target_stage.

```
// --- CHALLENGE 1: Listen for Leader Assignment ---
reflex receive_orders when: !empty(proposes) {
  loop msg over: proposes {
    // The Leader sends the assigned Stage object as the first content item
    list msgcontent <- msg.contents;
    target_stage <- Stage(msgcontent[0]);
  }
}
```

5.3 Demonstration

5.3.1 Initial Setting

There are a total of 50 guest agents, 4 stage agents and 1 leader agent. Yellow border means the guest loves the crowd and cyan means the guest hates the crowd.



5.3.2 Specific Guest Behavior ID 42

Here we observe a specific guest behavior (ID 42). Detailed Logs are written to the console. Sometimes the 42nd guest changes his mind.

We have identified 2 key scenarios. Both show the effectiveness of leader optimization.

Scenario 1: Crowd Relief (Cycle 501)

"Improving the experience without changing the location."

Guest 42 selects their favorite location, **Stage 1** (Base Score: 1.29). Initially, the stage is overcrowded with 18 agents, dragging Guest 42's utility down to 0.96 due to their high crowd aversion. During optimization, the Leader keeps Guest 42 at Stage 1 but successfully

disperses other agents to different stages. The population at Stage 1 drops to 9, raising Guest 42's final utility to **1.12** solely by reducing congestion.

```
--------------------------------------------------
 Guest 42 Analysis | Cycle: 501
   > Stage Evaluations:
     - Stage(0): 0.58
     - Stage(1): 1.29
     - Stage(2): 1.23
     - Stage(3): 0.56
GUEST DECISION: Stage(1)
Leader: Received all 50 messages. Starting optimization...
Assignment: Stage(1)
Stage Population: 18/50
Calculation: 1.29 (Max Base) + -0.92 * 0.36
Guest 42 Utility: 0.96
FINAL Assignment: Stage(1)
FINAL Stage Population: 9/50
FINAL Calculation: 1.29 (Max Base) + -0.92 * 0.18
FINAL Guest 42 Utility: 1.12
--------------------------------------------------
--------------------------------------------------
 Guest 42 Analysis | Cycle: 1001
   > Stage Evaluations:
     - Stage(0): 1.16
     - Stage(1): 1.1
     - Stage(2): 1.11
     - Stage(3): 0.73
GUEST DECISION: Stage(0)
Leader: Received all 50 messages. Starting optimization...
Assignment: Stage(0)
Stage Population: 26/50
Calculation: 1.16 (Max Base) + -0.92 * 0.52
Guest 42 Utility: 0.69
FINAL Assignment: Stage(1)
FINAL Stage Population: 4/50
FINAL Calculation: 1.1 (Max Base) + -0.92 * 0.08
FINAL Guest 42 Utility: 1.03
--------------------------------------------------
```

Scenario 2: Strategic Relocation (Cycle 1001)

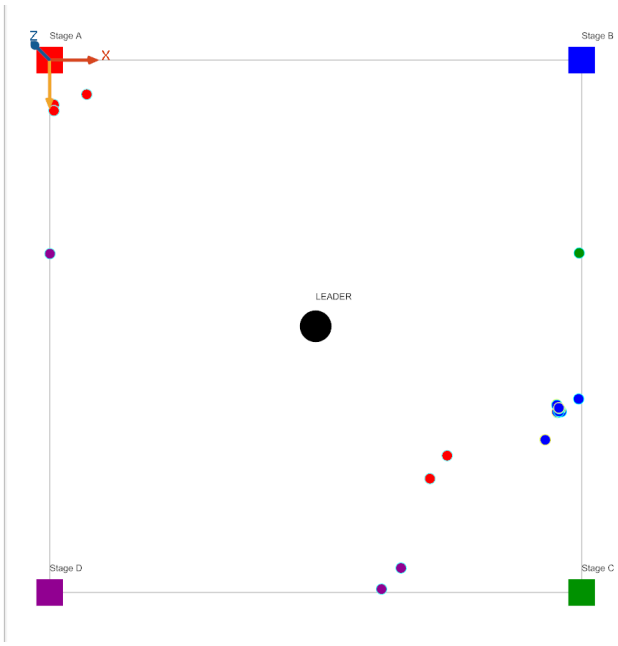
"Moving to a quiet runner-up is better than a crowded favorite."

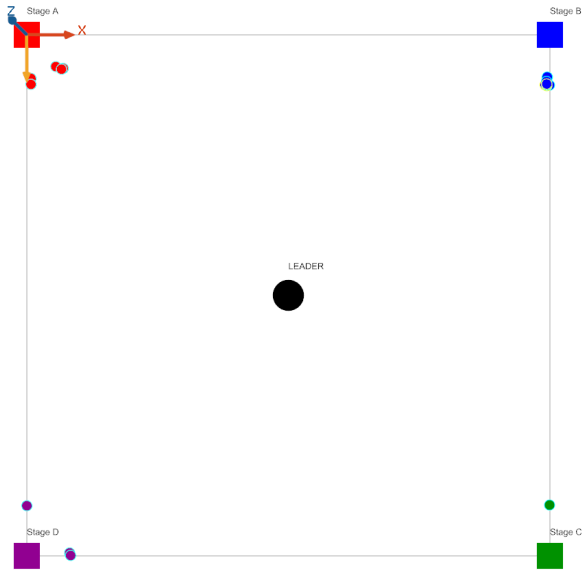
Guest 42 initially chooses their top preference, **Stage 0** (Base Score: 1.16). However, this stage is massively overcrowded (26 agents), resulting in a poor utility of 0.69. The Leader

recognizes this inefficiency and reassigns Guest 42 to their second choice, **Stage 1** (Base Score: 1.1). Although the base score is slightly lower, the stage is nearly empty (only 4 agents). This trade-off significantly increases Guest 42's final utility to **1.03**.

5.3.3 Global Map Visualisation

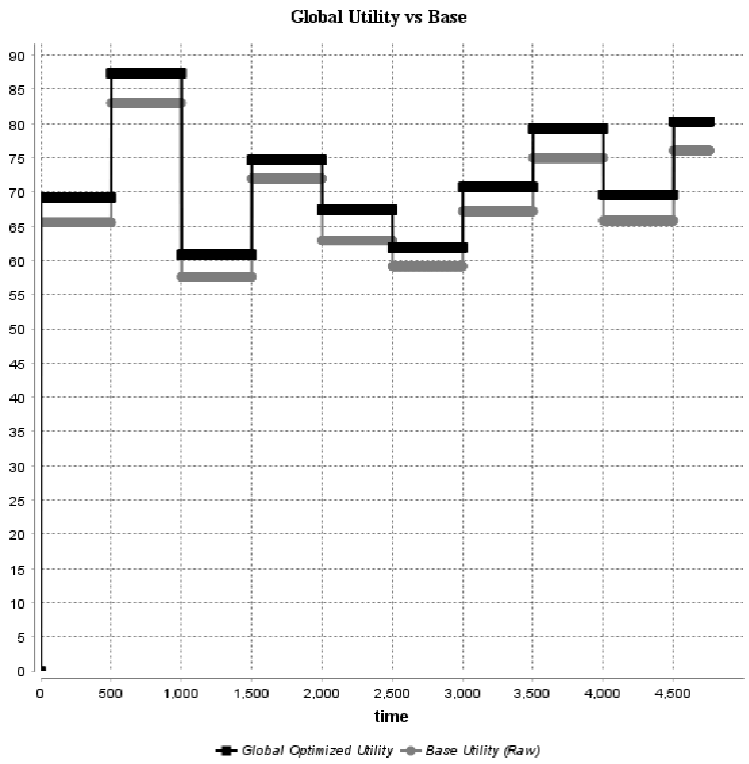
Similar to the case in task 2. We simulate guest behavior by changing their color to the stage that they want to go to, and every 500 cycles the stages reshuffle quality metrics and guests have to decide which stage to go to the next iteration.





5.3.3 Comparison of base utility and global utility after being optimized

As is shown in the line chart, thanks to the leader decision, the global utility is always optimized to be larger than the base utility.



Section 6: Conclusion

In this assignment, we enhanced our understanding of the GAMA platform by designing a multi-agent system that balances individual preferences with global optimization. We successfully implemented the FIPA interaction protocol to coordinate communication between Guests, Stages, and a Leader agent By integrating a Multi-Criteria Utility Analysis with a centralized hill-climbing algorithm, we demonstrated how agents can transition from selfish local decision-making to a socially optimized behavior that accounts for complex constraints like crowd density preferences.

One limitation of this model is the Leader's reliance on a basic hill-climbing heuristic, which, while effective for 50 agents, may struggle to find the true global maximum or suffer performance bottlenecks as the population scales. Additionally, the current logic assumes guests are fully compliant with the Leader's instructions. Future work could introduce "rebellious" behaviors where agents might negotiate or reject the Leader's proposal if their individual utility loss exceeds a certain threshold, simulating a more realistic trade-off between social compliance and personal satisfaction.